

SQL-Aufgaben zur Datenbank wws

Gruppe 1: Leicht (Grundlagen)

1. **Frage:** Zeige alle Spalten der Tabelle artikel an.
Lösung: `SELECT * FROM artikel;`
2. **Frage:** Liste alle Kundennamen (vorname, nachname) auf.
Lösung: `SELECT vorname, nachname FROM kunden;`
3. **Frage:** Zeige alle Artikel an, die mehr als 50 Euro kosten.
Lösung: `SELECT * FROM artikel WHERE preis > 50;`
4. **Frage:** Sortiere die Kategorien alphabetisch nach ihrem Namen.
Lösung: `SELECT * FROM kategorien ORDER BY kategorie_name ASC;`
5. **Frage:** Zeige die 5 günstigsten Artikel.
Lösung: `SELECT * FROM artikel ORDER BY preis ASC LIMIT 5;`
6. **Frage:** Suche alle Kunden, die aus "Berlin" kommen.
Lösung: `SELECT * FROM kunden WHERE ort = 'Berlin';`
7. **Frage:** Liste alle Bestellungen auf, die im Jahr 2023 getätigt wurden.
Lösung: `SELECT * FROM bestellungen WHERE bestelldatum LIKE '2023%';`
8. **Frage:** Zeige alle Artikel an, deren Bestand unter 10 Stück liegt.
Lösung: `SELECT * FROM artikel WHERE bestand < 10;`
9. **Frage:** Welche Artikel haben keinen Beschreibungstext (NULL)?
Lösung: `SELECT * FROM artikel WHERE beschreibung IS NULL;`
10. **Frage:** Liste alle Kunden auf, deren Nachname mit "M" beginnt.
Lösung: `SELECT * FROM kunden WHERE nachname LIKE 'M%';` (... weitere
11. **Frage:** Berechne den Lagerwert für jeden Artikel (Preis * Bestand).
Lösung: `SELECT artikel_name, (preis * bestand) AS lagerwert FROM artikel;`
12. **Frage:** Liste alle Kategorien auf, deren ID zwischen 2 und 5 liegt.
Lösung: `SELECT * FROM kategorien WHERE id BETWEEN 2 AND 5;`
13. **Frage:** Zeige alle Kunden, die keine E-Mail-Adresse hinterlegt haben.
Lösung: `SELECT * FROM kunden WHERE email IS NULL;`
14. **Frage:** Gib alle Artikel aus, deren Name das Wort "Bio" enthält.
Lösung: `SELECT * FROM artikel WHERE artikel_name LIKE '%Bio%';`

SQL-Aufgaben zur Datenbank wws

15. **Frage:** Berechne die Mehrwertsteuer (19%) für jeden Artikelpreis.

Lösung: `SELECT artikel_name, preis, (preis * 0.19) AS mwst
FROM artikel;`

16. **Frage:** Zeige die 10 neuesten Kunden (höchste ID).

Lösung: `SELECT * FROM kunden ORDER BY id DESC LIMIT 10;`

17. **Frage:** Liste alle Orte auf, aus denen Kunden kommen (ohne Duplikate).

Lösung: `SELECT DISTINCT ort FROM kunden;`

18. **Frage:** Zeige alle Bestellungen, die nach dem 01.01.2024 aufgegeben wurden.

Lösung: `SELECT * FROM bestellungen WHERE bestelldatum > '2024-01-01';`

19. **Frage:** Welche Artikel haben einen Bestand von exakt 0?

Lösung: `SELECT * FROM artikel WHERE bestand = 0;`

20. **Frage:** Gib die Artikelnamen in Großbuchstaben aus.

Lösung: `SELECT UPPER(artikel_name) FROM artikel;`

21. **Frage:** Sortiere die Kunden nach Ort (aufsteigend) und dann nach Nachname (absteigend).

Lösung: `SELECT * FROM kunden ORDER BY ort ASC, nachname DESC;`

Gruppe 2: Mittel (Joins, Aggregationen, GROUP BY, HAVING)

22. **Frage:** Zeige alle Artikelnamen zusammen mit ihrem Kategorienamen an.

Lösung: `SELECT a.artikel_name, k.kategorie_name FROM artikel a
JOIN kategorien k ON a.kategorie_id = k.id;`

23. **Frage:** Wie viele Artikel gibt es pro Kategorie?

Lösung: `SELECT kategorie_id, COUNT(*) FROM artikel GROUP BY
kategorie_id;`

24. **Frage:** Berechne den Durchschnittspreis aller Artikel.

Lösung: `SELECT AVG(preis) FROM artikel;`

25. **Frage:** Zeige alle Bestellungen mit dem dazugehörigen Kundennamen an.

Lösung: `SELECT b.id, k.vorname, k.nachname FROM bestellungen b
JOIN kunden k ON b.kunden_id = k.id;`

26. **Frage:** Welche Kunden haben mehr als 3 Bestellungen aufgegeben?

Lösung: `SELECT kunden_id, COUNT(*) FROM bestellungen GROUP BY
kunden_id HAVING COUNT(*) > 3;`

SQL-Aufgaben zur Datenbank wws

27. **Frage:** Liste die Bestellpositionen für die Bestell-ID 10 mit Artikelnamen auf.

Lösung: `SELECT bp.*, a.artikel_name FROM bestellpositionen bp
JOIN artikel a ON bp.artikel_id = a.id WHERE
bp.bestellungen_id = 10;`

28. **Frage:** Berechne den Gesamtumsatz (Summe aus Menge * Preis) über alle Bestellpositionen.

Lösung: `SELECT SUM(menge * verkaufspreis) FROM
bestellpositionen;`

29. **Frage:** Zeige die Kategorien an, in denen der Durchschnittspreis der Artikel über 100 Euro liegt.

Lösung: `SELECT k.kategorie_name, AVG(a.preis) FROM artikel a
JOIN kategorien k ON a.kategorie_id = k.id GROUP BY
k.kategorie_name HAVING AVG(a.preis) > 100;`

30. **Frage:** Welche Artikel wurden noch nie bestellt? (Nutze LEFT JOIN).

Lösung: `SELECT a.artikel_name FROM artikel a LEFT JOIN
bestellpositionen bp ON a.id = bp.artikel_id WHERE bp.id IS
NULL;`

31. **Frage:** Liste die Top 3 Kunden nach Umsatz.

Lösung: `SELECT k.nachname, SUM(bp.menge * bp.verkaufspreis) AS
umsatz FROM kunden k JOIN bestellungen b ON k.id =
b.kunden_id JOIN bestellpositionen bp ON b.id =
bp.bestellungen_id GROUP BY k.id ORDER BY umsatz DESC LIMIT
3;`

32. **Frage:** Ermittle für jede Bestellung die Anzahl der unterschiedlichen Artikelpositionen.

Lösung: `SELECT bestellungen_id, COUNT(artikel_id) FROM
bestellpositionen GROUP BY bestellungen_id;`

33. **Frage:** Zeige den Namen des Kunden und das Datum seiner letzten Bestellung.

Lösung: `SELECT k.nachname, MAX(b.bestelldatum) FROM kunden k
JOIN bestellungen b ON k.id = b.kunden_id GROUP BY k.id;`

34. **Frage:** Liste alle Artikel auf, die zur Kategorie 'Obst' gehören (Name der Kategorie nutzen).

Lösung: `SELECT a.* FROM artikel a JOIN kategorien kat ON
a.kategorie_id = kat.id WHERE kat.kategorie_name = 'Obst';`

SQL-Aufgaben zur Datenbank wws

35. **Frage:** Berechne die Gesamtzahl der verkauften Einheiten für den Artikel mit der ID 5.

Lösung: `SELECT SUM(menge) FROM bestellpositionen WHERE artikel_id = 5;`

36. **Frage:** Welche Kategorien enthalten mehr als 50 verschiedene Artikel?

Lösung: `SELECT kategorie_id FROM artikel GROUP BY kategorie_id HAVING COUNT(*) > 50;`

37. **Frage:** Zeige alle Bestellungen (ID und Datum) und die Anzahl der darin enthaltenen Artikelpositionen.

Lösung: `SELECT b.id, b.bestelldatum, COUNT(bp.id) FROM bestellungen b LEFT JOIN bestellpositionen bp ON b.id = bp.bestellungen_id GROUP BY b.id;`

38. **Frage:** Liste alle Kunden auf, die in der gleichen Stadt wohnen wie der Kunde mit der ID 1.

Lösung: `SELECT * FROM kunden WHERE ort = (SELECT ort FROM kunden WHERE id = 1) AND id <> 1;`

39. **Frage:** Berechne den durchschnittlichen Verkaufspreis pro Bestellung.

Lösung: `SELECT bestellungen_id, AVG(verkaufspreis) FROM bestellpositionen GROUP BY bestellungen_id;`

40. **Frage:** Zeige alle Artikel an, deren Preis über dem Gesamtdurchschnitt aller Artikel liegt.

Lösung: `SELECT * FROM artikel WHERE preis > (SELECT AVG(preis) FROM artikel);`

41. **Frage:** Welcher Artikel wurde am häufigsten (nach Menge) verkauft?

Lösung: `SELECT artikel_id, SUM(menge) as gesamtmenge FROM bestellpositionen GROUP BY artikel_id ORDER BY gesamtmenge DESC LIMIT 1;`

42. **Frage:** Erstelle eine Liste aller Kunden, die im März bestellt haben.

Lösung: `SELECT DISTINCT k.* FROM kunden k JOIN bestellungen b ON k.id = b.kunden_id WHERE MONTH(b.bestelldatum) = 3;`

SQL-Aufgaben zur Datenbank wws

Gruppe 3: Schwer (Subqueries, Views, komplexe Logik, DCL/DDDL)

43. **Frage:** Erstelle eine View v_kundenumsatz, die den Gesamtumsatz pro Kunde zeigt.

Lösung: CREATE VIEW v_kundenumsatz AS SELECT k.id, k.nachname, SUM(bp.menge * bp.verkaufspreis) AS gesamtumsatz FROM kunden k JOIN bestellungen b ON k.id = b.kunden_id JOIN bestellpositionen bp ON b.id = bp.bestellungen_id GROUP BY k.id;

44. **Frage:** Finde alle Artikel, deren Preis höher ist als der Durchschnittspreis ihrer eigenen Kategorie.

Lösung: SELECT artikel_name, preis FROM artikel a1 WHERE preis > (SELECT AVG(preis) FROM artikel a2 WHERE a1.kategorie_id = a2.kategorie_id);

45. **Frage:** Zeige den Kunden an, der die zeitlich aktuellste Bestellung getätigt hat, ohne LIMIT zu nutzen.

Lösung: SELECT * FROM kunden WHERE id = (SELECT kunden_id FROM bestellungen WHERE bestelldatum = (SELECT MAX(bestelldatum) FROM bestellungen));

46. **Frage:** Erstelle einen neuen Benutzer 'buchhaltung' und gib ihm nur Leserechte auf die View v_kundenumsatz.

Lösung: CREATE USER 'buchhaltung'@'localhost' IDENTIFIED BY 'Passwort123'; GRANT SELECT ON v_kundenumsatz TO 'buchhaltung'@'localhost';

47. **Frage:** Erhöhe den Bestand aller Artikel in der Kategorie 'Elektronik' um 10%.

Lösung: UPDATE artikel SET bestand = bestand * 1.1 WHERE kategorie_id = (SELECT id FROM kategorien WHERE kategorie_name = 'Elektronik');

48. **Frage:** Liste alle Monate des Jahres 2023 auf, in denen der Umsatz über 5000 Euro lag.

Lösung: SELECT MONTH(b.bestelldatum) as monat, SUM(bp.menge * bp.verkaufspreis) FROM bestellungen b JOIN bestellpositionen bp ON b.id = bp.bestellungen_id WHERE YEAR(b.bestelldatum) = 2023 GROUP BY monat HAVING SUM(bp.menge * bp.verkaufspreis) > 5000;

SQL-Aufgaben zur Datenbank wws

49. **Frage:** Finde Kunden, die Artikel aus JEDER verfügbaren Kategorie bestellt haben.

Lösung: SELECT k.nachname FROM kunden k JOIN bestellungen b ON k.id = b.kunden_id JOIN bestellpositionen bp ON b.id = bp.bestellungen_id JOIN artikel a ON bp.artikel_id = a.id GROUP BY k.id HAVING COUNT(DISTINCT a.kategorie_id) = (SELECT COUNT(*) FROM kategorien);

50. **Frage:** Nutze eine CASE-Anweisung, um Artikel in 'Günstig' (<20), 'Mittel' (20-100) und 'Premium' (>100) zu klassifizieren.

Lösung: SELECT artikel_name, preis, CASE WHEN preis < 20 THEN 'Günstig' WHEN preis BETWEEN 20 AND 100 THEN 'Mittel' ELSE 'Premium' END AS klasse FROM artikel;

51. **Frage:** Lösche alle Kunden, die seit mehr als 5 Jahren keine Bestellung mehr getätigt haben.

Lösung: DELETE FROM kunden WHERE id NOT IN (SELECT kunden_id FROM bestellungen WHERE bestelldatum > DATE_SUB(NOW(), INTERVAL 5 YEAR));

52. **Frage:** Erstelle eine Abfrage, die den prozentualen Anteil jeder Kategorie am Gesamtlagerwert ermittelt.

Lösung: SELECT k.kategorie_name, SUM(a.preis * a.bestand) / (SELECT SUM(preis * bestand) FROM artikel) * 100 AS prozent_anteil FROM artikel a JOIN kategorien k ON a.kategorie_id = k.id GROUP BY k.kategorie_name;

53. **Frage:** Erstelle eine Abfrage, die für jede Kategorie den teuersten Artikel ausgibt (Name und Preis).

Lösung: SELECT a.artikel_name, a.preis, k.kategorie_name FROM artikel a JOIN kategorien k ON a.kategorie_id = k.id WHERE a.preis = (SELECT MAX(preis) FROM artikel WHERE kategorie_id = k.id);

54. **Frage:** Berechne den "Bestandswert" (Bestand * Preis) pro Kategorie und zeige nur Kategorien mit einem Wert > 10.000 €.

Lösung: SELECT k.kategorie_name, SUM(a.bestand * a.preis) AS lagerwert FROM artikel a JOIN kategorien k ON a.kategorie_id = k.id GROUP BY k.kategorie_name HAVING lagerwert > 10000;

55. **Frage:** Finde alle Kunden, die seit ihrer Registrierung noch nie etwas bestellt haben (Subquery-Variante).

Lösung: SELECT * FROM kunden WHERE id NOT IN (SELECT DISTINCT kunden_id FROM bestellungen);

SQL-Aufgaben zur Datenbank wws

56. **Frage:** Erstelle einen Index auf die Spalte `artikel_name`, um die Suche zu beschleunigen.

Lösung: `CREATE INDEX idx_artikel_name ON
artikel(artikel_name);`

57. **Frage:** Gib eine Liste aus: Kundenname und die Info 'Stammkunde' (wenn > 5 Bestellungen) oder 'Neukunde'.

Lösung: `SELECT nachname, CASE WHEN (SELECT COUNT(*) FROM
bestellungen WHERE kunden_id = k.id) > 5 THEN 'Stammkunde'
ELSE 'Neukunde' END AS status FROM kunden k;`

58. **Frage:** Aktualisiere den Verkaufspreis in `bestellpositionen` auf den aktuellen Preis aus der Tabelle `artikel` für alle offenen Bestellungen (Annahme: Preisänderung nachpflegen).

Lösung: `UPDATE bestellpositionen bp JOIN artikel a ON
bp.artikel_id = a.id SET bp.verkaufspreis = a.preis;`

59. **Frage:** Zeige den kumulierten Umsatz pro Monat für das gesamte Jahr 2023.

Lösung: `SELECT MONTH(bestelldatum) AS monat, SUM(menge *
verkaufspreis) OVER (ORDER BY MONTH(bestelldatum)) AS
laufender_umsatz FROM bestellungen b JOIN bestellpositionen
bp ON b.id = bp.bestellungen_id WHERE YEAR(bestelldatum) =
2023;`

60. **Frage:** Erstelle eine Tabelle `archiv_bestellungen` mit der gleichen Struktur wie `bestellungen`.

Lösung: `CREATE TABLE archiv_bestellungen LIKE bestellungen;`

61. **Frage:** Finde die Artikel, die in mehr als 50% aller Bestellungen vorkommen.

Lösung: `SELECT artikel_id FROM bestellpositionen GROUP BY
artikel_id HAVING COUNT(DISTINCT bestellungen_id) > (SELECT
COUNT(*) FROM bestellungen) / 2;`

62. **Frage:** Entziehe dem Benutzer 'gast' das Recht, Daten in der Tabelle `artikel` zu löschen.

Lösung: `REVOKE DELETE ON artikel FROM 'gast';`